

CAMADA FÍSICA DA COMPUTAÇÃO

ENGENHARIA DA COMPUTAÇÃO - Rodrigo Carareto

PROJETO 3 – PROTOCOLO DE TRANSMISSÃO DE ARQUIVOS ATRAVÉS DE DATAGRAMAS

INTRODUÇÃO TEÓRICA

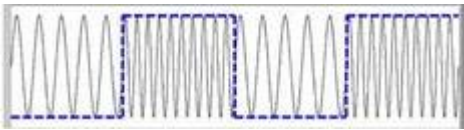
Normalmente, a camada responsável pela transmissão dos dados gerados em camadas superiores, realiza a transmissão de duas formas: dados fragmentados em pacotes (datagramas) ou streaming. As principais diferenças entre uma transmissão em streaming e uma transmissão baseada em pacotes (datagramas) estão relacionadas à forma como os dados são enviados, recebidos e processados. Vamos analisar cada uma:

Transmissão em Streaming

- Definição: É uma forma de transmissão contínua de dados, geralmente usada para conteúdos multimídia (áudio e vídeo).
- Protocolo comum: Utiliza protocolos como RTP (Real-time Transport Protocol) sobre UDP ou HLS/DASH sobre HTTP.
- Modo de entrega: Os dados são entregues e processados continuamente, permitindo a reprodução do conteúdo quase em tempo real, sem necessidade de download completo.
- Latência: Baixa a moderada, dependendo da técnica utilizada (por exemplo, buffering para evitar interrupções).
- Controle de erro: Pode tolerar pequenas perdas sem comprometer significativamente a experiência do usuário.
- Exemplos de uso: Plataformas como Netflix, YouTube, Spotify e transmissões ao vivo.

Transmissão em Pacotes (Datagramas)

- Definição: Baseia-se no envio de pacotes de dados independentes, onde cada datagrama pode seguir um caminho diferente na rede.
- Exemplos: Protocolos como UDP (User Datagram Protocol), que não garante a entrega ou a ordem dos pacotes. E protocolos como o TCP (Transmission Control Protocol), que provê garantia de entrega.
- Modo de entrega: Os pacotes são enviados de forma discreta, sem garantir que chegarão na mesma ordem ou mesmo que serão entregues (dependendo do protocolo).
- Latência: Aumenta quando há necessidade de confirmação de recebimento de pacotes, como no TCP.
- Controle de erro: Garantia de retransmissão no caso do TCP.
- Exemplos de uso: Aplicações em tempo real como VoIP (ligações pela internet), jogos online, DNS, transmissões multicast.



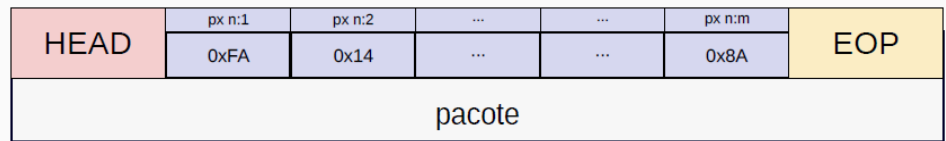
CAMADA FÍSICA DA COMPUTAÇÃO

ENGENHARIA DA COMPUTAÇÃO - Rodrigo Carareto

Como são os pacotes (datagramas)

Um datagrama é tipicamente dividido em 3 partes:

- Um cabeçalho (header)
- Payload (dados)
- EOP (end of package)]



Nos bytes reservados ao cabeçalho, são colocados os bytes de comunicação entre as partes. O metadado. Com os bytes posicionados nas posições corretas, as partes podem conversar. Confirmar envio, informar quem está enviando, qual o número do pacote, qual o tamanho do payload, confirmar recebimento... Enfim, esses bytes são utilizados para uma conversa entre as partes.

No payload são colocados os bytes de dados que foram acomodados no pacote.

O EOP é uma espécie de fim de pacotes, é uma sequência combinada que marca o fim do pacote.

Por que enviar os dados segmentados em pacotes?

Os principais motivos para o uso de transmissão em pacotes são:

1. Eficiência na utilização da rede

- Dividir dados em pacotes permite que várias transmissões ocorram simultaneamente na rede.
- Diferentes pacotes podem seguir **rotas distintas** para evitar congestionamentos.
- Se um pacote for perdido, **somente ele será retransmitido**, e não toda a mensagem.

2. Melhor controle de erro

- Cada pacote contém um **checksum** para verificar a integridade dos dados.
- Se houver erro na transmissão, **apenas o pacote corrompido será reenviado**, reduzindo desperdício de banda.

3. Gerenciamento de tráfego e controle de congestionamento

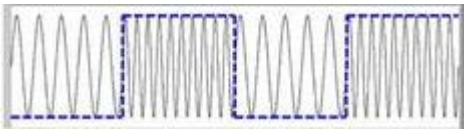
- Protocolos como **TCP** ajustam o tamanho dos pacotes e a taxa de envio conforme a capacidade da rede.
- Se a rede estiver congestionada, pacotes podem ser **redirecionados** ou **retransmitidos**.

4. Fragmentação e reassemblagem

- Pacotes menores são mais fáceis de transportar e evitam **bloqueios** em redes de diferentes capacidades.
- Dispositivos intermediários, como roteadores, podem **fragmentar ou juntar pacotes** conforme necessário.

5. Comunicação mais robusta e confiável

- Se uma conexão for interrompida, pacotes já transmitidos **não são perdidos**.
- A comunicação pode ser mantida mesmo com falhas parciais na rede.



CAMADA FÍSICA DA COMPUTAÇÃO

ENGENHARIA DA COMPUTAÇÃO - Rodrigo Carareto

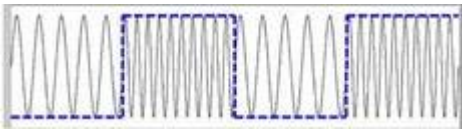
Exemplo de datagrama: Cabeçalho TCP (20 bytes fixos + opções variáveis)

Campo	Tamanho (bits)	Descrição
Porta de origem	16	Número da porta do remetente
Porta de destino	16	Número da porta do destinatário
Número de sequência	32	Indica a posição do primeiro byte deste segmento dentro do fluxo de dados
Número de confirmação (ACK)	32	Confirma o recebimento do último segmento válido do emissor
Tamanho do cabeçalho	4	Indica o tamanho do cabeçalho TCP (mínimo 20 bytes)
Reservado	3	Reservado para uso futuro
Flags de controle	9	Indica o estado da conexão (SYN, ACK, FIN, etc.)
Tamanho da janela	16	Quantidade de bytes que o receptor pode aceitar sem receber confirmação
Checksum	16	Verifica a integridade do segmento
Ponteiro de urgência	16	Indica se há dados urgentes (caso a flag URG esteja ativa)
Opções TCP	Variável	Usado para configurações extras, como escala de janela, timestamps, etc.

Além do cabeçalho, o protocolo tem um payload variável, com tamanho máximo de 1469 bytes. O tamanho do payload de cada pacote é normalmente informado no cabeçalho.

Explicação dos principais campos do cabeçalho TCP

- Número de sequência (Sequence Number)**
 - Indica qual é o primeiro byte do segmento dentro do fluxo de dados.
 - Importante para remontar os dados na ordem correta.
- Número de confirmação (Acknowledgment Number)**
 - Usado pelo destinatário para informar qual o próximo byte esperado.
 - Se um segmento for perdido, ele não será confirmado, e o remetente o reenviará.
- Flags de controle (9 bits)**
 - URG** (Urgent) → Indica dados urgentes.



CAMADA FÍSICA DA COMPUTAÇÃO

ENGENHARIA DA COMPUTAÇÃO - Rodrigo Carareto

- **ACK** (Acknowledgment) → Confirma recebimento de dados.
- **PSH** (Push) → Solicita entrega imediata ao aplicativo.
- **RST** (Reset) → Reinicia a conexão abruptamente.
- **SYN** (Synchronize) → Inicia uma conexão.
- **FIN** (Finish) → Finaliza uma conexão.

4. Tamanho da Janela (Window Size)

- Define quantos bytes o receptor pode armazenar antes de precisar enviar uma confirmação.
- Essencial para o **controle de congestionamento** e **fluxo de dados**.

5. Checksum

- Validado pelo receptor para garantir que os dados não foram corrompidos durante a transmissão.

Enunciado do projeto 03

Nesse projeto, sua aplicação que exerce o papel de client deverá realizar download de arquivos vindos da aplicação server. Esses arquivos deverão ser fragmentados e enviados através de “pacotes” (datagramas). Deverá existir handshake e confirmação de recebimento de cada pacote, como explicado a seguir!

Datagrama.

Voce pode definir como será seu datagrama, mas aconselhamos a reservar uma boa quantidade de bytes (10 ou 12 bytes) para o head, a fim de ter espaço suficiente para uma boa comunicação entre as partes. O tamanho do head deve ser fixo. O payload pode ter tamanho variável durante a transmissão, mas **nunca poderá ser maior que 100 Bytes**. Além disso sugerimos um EOP de sua livre escolha com 3 ou 4 bytes.

Handshake

Antes do início do envio da mensagem, o client deve enviar uma mensagem para verificar se o server está “vivo” e indagar quais arquivos estão disponíveis. O server deve responder com os nomes dos arquivos disponíveis, e esses nomes deverão aparecer no terminal do cliente.

Escolha de 2 ou mais arquivos.

O client então solicita um dos arquivos. O server deve responder algo como: “arquivo x escolhido, deseja adicionar outro arquivo?”

O cliente deverá então escolher um segundo arquivo.

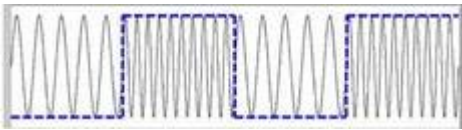
O server deve responder com “arquivo x e y escolhidos, deseja adicionar outro arquivo?”

Se o client responder que sim, o server fornece a lista disponível, o cliente escolhe outro arquivo, e o server vai registrando as escolhas. Se o cliente responder que não, o server deve responder algo como: “entendido. Vou iniciar a transmissão simultânea dos arquivos escolhidos”

Transmissão

A transmissão deve ser necessariamente feita da seguinte maneira:

Toda vez que o server enviar um pacote de um dos arquivos, o cliente deve responder confirmando o recebimento, e só então o server envia o próximo. Todas essas confirmações devem estar sendo *printadas* no terminal. Descreva tudo que esteja ocorrendo através de prints.



CAMADA FÍSICA DA COMPUTAÇÃO

ENGENHARIA DA COMPUTAÇÃO - Rodrigo Carareto

Os pacotes devem ser alternados, ou seja, um pacote de cada arquivo, sendo que os dois ou mais arquivos sejam transmitidos simultaneamente enquanto a transmissão não termina. À medida que arquivos sejam enviados completamente, apenas os pacotes dos arquivos ainda em transmissão são enviados, obviamente.

Fim

Os arquivos devem ser salvos no computador do cliente e não podem estar corrompidos.

Um print com o resumo da transmissão deve aparecer, mostrando tamanho dos arquivos recebidos, número de pacotes de cada arquivo.

FUNCIONALIDADES DE ENTREGA:

Você deverá mostrar até 03/09/2026

Funcionalidades obrigatórias:

- Uma transmissão de sucesso de 2 arquivos simultâneos escolhidos pelo usuário.
- Transmissão de sucesso mais a funcionalidade em que o usuário client poderá apartar uma tecla para pausar o processo, uma outra tecla para reiniciar o processo e outra para abortar a transmissão.
- Uma simulação em que se os fios entre os Arduinos (qualquer um deles ou ambos) forem desconectados e conectados novamente, a transmissão retorna e termina com sucesso!
- Todas as funcionalidades com prints caprichados de todo o processo no lado cliente e também server!

Dica! Lembre-se sempre que server e client podem conversar autonomamente através de mensagens no head. Cada byte em cada posição do head pode significar algo! Além disso, o time out pode ser usado para tomada de decisões caso algo esperado não ocorra... Criem seu próprio protocolo de download baseado em datagramas!